

BMS2.0 协议文档

基于ycprotocol

修改记录

修改日期	修改人	修改内容
2024/11/12	zhangyf03	增加： 1. cmdGroup=0x02：命令号 02-07； 2. cmdGroup=0x03：命令号 07-21； 3. cmdGroup=0x04：命令号 04-05；
2024/11/18	yuch	转发UPS、温湿度传感器数据 命令调整到M核
2024/12/11	yuch	转发命令数据体填写，并将“转发UPS、温湿度传感器数据”根据需求改为“转发UPS、温湿度传感器数据“
2024/12/20	yuch	增加BBMS_M DIDO状态命令
2024/12/24	yuch	把02\08命令的描述改成转发外部设备信息，添加TMS数据
2024/12/24	zhangyf03	cmdGroup=0x02，命令号 02 重命名：TMS_Info -> TMS_SumInfo
2025/01/03	luy	增加RBMS DIDO状态命令
2025/01/13	zhangyf03	增加： 1. cmdGroup=0x02，命令号 10：HMI三级板DO控制命令； 2. cmdGroup=0x03， a. 命令号 23：PCB板温信息； b. 命令号 24：HMI二级板DO控制命令； 修改：cmdGroup=0x03，命令号 09 重命名为 HMI_RackDOCtrl -> HMI_RBMSRlyCtrl；
2025/02/18	zhangyf03	增加：cmdGroup=0x03, 1. 命令号 25：SOX算法调试的输入数据； 2. 命令号 26：SOX算法调试的输出数据；
2025/03/05	yuch	增加 cmdGroup=0x04,命令号 0x06 清除板载NVM数据
2025/03/11	zhangyf03	增加 cmdGroup=0x03，命令号27：Rack中每个AFE的总压

2025/03/21	zhangyf03	增加： 1. cmdGroup=0x03, 命令号28-29：Rack故障累积次数标定； 2. cmdGroup=0x02, 命令号11-12：Bank故障累积次数标定；
2025/03/28	zhangyf03	删除：cmdGroup=0x03, 命令号29；cmdGroup=0x02, 命令号12；
2025/04/01	zhangyf03	增加 cmdGroup=0x03, 命令号29：极柱温度；
2025/04/07	yuch	增加 cmdGroup=0x04, 命令号7：启动/禁止设备主动报文；
2025/04/09	luy	增加： 1. cmdGroup=0x05, 命令号1：上位机发送给主动均衡板位置命令； 2. cmdGroup=0x05, 命令号2：主动均衡板发送给上位机位置状态；
2025/04/09	luy	增加： 1. cmdGroup=0x03, 命令号30：二级板发送给上位机的需要均衡位置信息
2025/04/18	luy	增加： 1. cmdGroup=0x03, 命令号31：二级板发送给上位机的需要均衡位置信息 2. cmdGroup=0x05, 命令号3,4,5,6,：二级板与主动均衡板之间交互指令
2025/05/12	zhangyf03	增加 cmdGroup=0x03, 命令号32：Pack正负极连接件温度；
2025/05/14	zhangyf03	删除 cmdGroup=0x04, 命令号5：HMI Logger广播信号；
2025/05/30	zhangyf03	变更： 1. cmdGroup=0x03, 命令号2：RBMS_Volt； 2. cmdGroup=0x03, 命令号3：RBMS_Temp； 3. cmdGroup=0x04, 命令号5：HMI_FltOvTiNbr； 删除：cmdGroup=0x03, 命令号23、27、29、32；
2025/06/11	zhangyf03	增加： 1. cmdGroup=0x02, 命令号12：BBMS_CtlWordAllRack； 2. cmdGroup=0x03, 命令号23：RBMS_Debug；
2025/06/20	yuch	新增： 1、命令组=0x01, 命令号3 写入堆电池生产信息 ↩ 2、命令组=0x01, 命令号4 读取堆电池生产信息 3、命令组=0x01, 命令号5 设置EMS 端口IP ↩ 4、命令组=0x01, 命令号6 读取EMS 端口IP

2025/07/10	luy	新增： 1、命令组=0x03，命令号33 切换CAN0 PACK下线测试模式
2025/07/14	zhangyf03	新增： 1. cmdGroup=0x01，命令号7：单芯计算的SOC以及各簇SOC结果； 2. cmdGroup=0x01，命令号8：单芯计算的SOH以及各簇SOH结果； 3. cmdGroup=0x01，命令号9：BBMS_A核的故障列表；
2025/07/29	yuch	新增：↵ 1、cmdGroup=0x02，命令号13：掉电通知
2025/08/04	yuch	修改：↵ 1、命令组=0x01，命令号3 写入堆电池生产信息改为写入堆电池生产日期 ↵ 2、命令组=0x01，命令号4 读取堆电池生产信息改为读取堆电池生产日期
2025/08/05	zhangyf03	新增： 1. cmdGroup=0x01，命令号10：自放电率演变信息； 2. cmdGroup=0x04，命令号8：读取-故障使能数组； 3. cmdGroup=0x04，命令号9：写入-故障使能数组；
2025/08/07	zhangyf03	删除：cmdGroup=0x02，命令号5：Bank累计充放电容量能量标定；
2025/08/21	zhangyf03	新增：cmdGroup=0x02，命令号14：功能安全相关信号
2025/08/27	yuch	修改：↵ 1、命令组=0x01，命令号5 添加3个保留字节数据对齐
2025/09/03	yuch	新增：↵ 1、命令组=0x01，命令号11 NTP服务器IP设置
2025/09/16	yuch	新增：↵ 1、命令组3、命令号34 二级板上报外设数据 ↵ 修改：↵ 1、命令组2、命令号8 三级板上报外设数据（改为主动上报）
2025/10/14	yuch	增加HIL工具节点地址
2025/10/20	yuch	新增：↵ 1、命令组=0x01，命令号12 13 时区设置和读取
2025/10/27	lik03	新增二级板日志打印等级，便于现场排查问题： 命令组3，命令号35
2025/11/05	yuch	增加自动化工具节点地址

2025/12/01	zhangyf03	新增： 1. cmdGroup=0x03, 命令号37：HMI TMS控制指令(RBMS)； 2. cmdGroup=0x03, 命令号38：热管理信息(RBMS)； 3. cmdGroup=0x03, 命令号39-40：读取/写入-TMS阈值参数(RBMS)；
2026/01/13	lik03	新增： 1. cmdGroup=0x04, 命令号10：设置二、三级板的子网掩码； 2. cmdGroup=0x04, 命令号11：读取二、三级板的子网掩码；
2026/01/13	xiaow	新增： 1. cmdGroup=0x01, 命令号14：设置三级板EMS 端口的子网掩码； 2. cmdGroup=0x01, 命令号15：读取三级板EMS 端口的子网掩码；
2026/01/21	xiaow	新增： 1. cmdGroup=0x01, 命令号16：新增三级板A核控制字；
2026/01/23	xiaow	新增 1. cmdGroup = 0x02, 命令号15：设置当前接入的除湿机485通讯地址； 2. cmdGroup = 0x02, 命令号16：读取当前接入的除湿机485通讯地址；
2026/02/02	xiaow	新增 1. cmdGroup = 0x01, 命令号17：设置三级板打印日志级别； 2. cmdGroup = 0x01, 命令号18：读取三级板打印日志级别； 3. cmdGroup = 0x01, 命令号19：发送A核RTC时间
2026/02/25	xiaow	新增 1.cmdGroup = 0x02, 命令号17, A核设置M核打印日志级别（仅在调试M核程序时使用，RS485_4被占用时（目前RS485_4外接电表或UPS)无法使用）； 2.cmdGroup = 0x02, 命令号18, A核读取M核打印日志级别（仅在调试M核程序时使用，RS485_4被占用时（目前RS485_4外接电表或UPS)无法使用）。
2026/03/12	xiaow	新增 1.cmdGroup = 0x01, 命令号20, 获取A核系统运行参数。
2026/03/13	zhangyf03	新增： 1. cmdGroup = 0x03, 命令号41：发送RBMS_Fault； 2. cmdGroup = 0x02, 命令号19：发送BBMS_Fault； 删除：cmdGroup = 0x04, 命令号01：原RBMS_Fault、BBMS_Fault结构体

2026/03/30	xiaow	新增 1.cmdGroup = 0x02, 命令号20：设置M核CAN报文是否加密（由A核发送命令）； 2.cmdGroup = 0x02, 命令号21：获取M核CAN报文是否加密（由A核发送命令）； 3.cmdGroup = 0x01, 命令号21：设置A核报文是否加密并写入指定文件（由M核发送命令）； 4.cmdGroup = 0x01, 命令号22：从指定文件获取A核报文是否加密选项值（由M核发送命令）。
2026/04/15	wandp	新增 1.cmdGroup = 0x01, 命令号23：设置N拖M三级板ID号（只支持N托M架构使用）； 2.cmdGroup = 0x01, 命令号24：获取N拖M三级板ID号； 3.cmdGroup = 0x01, 命令号25：设置三级板主机/从机模式（只支持N托M架构使用）； 4.cmdGroup = 0x01, 命令号26：获取三级板（主机/从机）模式状态； 5.cmdGroup = 0x01, 命令号27：设置预制舱编号(箱号)； 6.cmdGroup = 0x01, 命令号28：获取预制舱编号(箱号)；
2026/04/23	wandp	新增 1.cmdGroup = 0x01, 命令号29：设置61850客户指定名称功能； 2.cmdGroup = 0x01, 命令号30：获取61850客户指定名称功能； 修改 1.cmdGroup = 0x01, 命令号11：NTP IP设置支持双通道，0x01代表A网和0x02代表B网
2026/04/23	xiaow	1.三级板M核cmdGroup = 0x02, 命令号8外设数据报文结构体新增EMS点表地址和外设ID号
2026/05/09	xiaow	修改 1.cmdGroup = 0x01, 命令号20, 新增获取A核核间通讯发送/接收速率和PHY复位次数。
2026/05/13	xiaow	修改 1.cmdGroup = 0x01, 命令号11, 新增NTP IP的删除功能； 新增 2.cmdGroup = 0x01, 命令号31, 新增NTP IP的回读功能。
2026/05/15	xiaow	新增 1.cmdGroup = 0x01, 命令号32, 新增NTP IP联通测试功能。

2026/05/20	xiaow	新增 1.cmdGroup = 0x01，命令号33，新增M核系统运行状态命令(由M核定时向A核发送)； 2.cmdGroup = 0x02，命令号22，新增A核系统运行状态命令(由A核定时向M核发送)。
2026/05/26	xiaow	修改 1.cmdGroup = 0x01，命令号20，新增获取当前A核与二级板以太网成功连接簇数、当前A核与二级板以太网成功连接状态、A核检测二级板心跳状态和M核检测二级板心跳状态。

设备节点

设备类型	主节点号	子节点号
广播	0x00	0
上位机与A核组播	0x01	0
上位机（电脑）	0x01	1
三级板A核	0x01	2
三级板A核OTA进程	0x01	3
HIL工具	0x01	4
自动化工具	0x01	5
三级板M核	0x03	1
二级板	0x04	N
主动均衡板	0x05	N

命令数据

不需要响应的数据，命令写在发送方

需要响应的报文，命令定义在响应方

二、BMS2.0 BBMS_A业务传输命令（cmdGroup = 0x01）

三级板A核需要接收并处理这些数据

命令号	命令类型	功能	请求payload	响应payload
01	NEED_ACK	写入参数	{ char name[10]; uint8_t data[]; //不定长数组 }	{ uint8_t state; //00：写入成功，01：写入失败 }
02	NEED_ACK	读取参数	{ char name[n]; //n最大是10 }	{ uint32_t len; //当查询为空时返回长度0 uint8_t data[]; //不定长数组 }
03	NEED_ACK	写入堆电池生产信息	{ uint8 year; //年份为后两位2025年为25 uint8 month; ← uint8 day; }	{ uint8_t state; //00：写入成功，01：写入失败 }
04	NEED_ACK	读取堆电池生产信息		{ uint8 year; //年份为后两位2025年为25 uint8 month; ← uint8 day; }
05	NEED_ACK	设置EMS 端口IP	struct in_addr { //小端字序 uint32_t s_addr; // IPv4 address (in network byte order) uint8_t port; //1:ETH1, 2:ETH2, 3:EMS1, 4:EMS2 uint8_t reserved[3]; //数据对齐 };	struct state{ uint8_t state; //00：写入成功，01：写入失败 }
06	NEED_ACK	读取EMS 端口IP	struct port { uint8_t port; //1:ETH1, 2:ETH2, 3:EMS1, 4:EMS2 };	struct in_addr { //小端字序 uint32_t s_addr; };
07	UNNEED_ACK	单芯计算的SOC以及各簇SOC结果	struct BBMS_A_SOCInfo	
08	UNNEED_ACK	单芯计算的SOH以及各簇SOH结果	struct BBMS_A_SOHInfo	
09	UNNEED_ACK	BBMS_A核的故障列表	struct FaultFlags	
10	UNNEED_ACK	自放电率演变信息		struct BBMS_A_Selfdr

11	NEED_ACK	设置NTP服务器IP	<pre> struct in_addr { uint8_t cmd; //1：增加 2：删除 //小端字序 // IPv4 subnet_address (in network byte order) uint32_t s_addr; //要操作的IP地址 }; </pre>	<pre> struct state{ uint8_t state; //00：操作成功，01：操作失败 } </pre>
12	NEED_ACK	设置时区	<pre> int8_t timeZone; //0：UTC，8：表示东8区，-8：表示西8区 </pre>	
13	NEED_ACK	读取时区		<pre> int8_t timeZone; //0：UTC，8：表示东8区，-8：表示西8区 </pre>
14	NEED_ACK	设置EMS 端口子网掩码	<pre> struct in_addr { //小端字序 uint32_t s_addr; // IPv4 subnet_address (in network byte order) uint8_t port; //1:ETH1, 2:ETH2, 3:EMS1, 4:EMS2 uint8_t reserved[3]; //数据对齐 }; </pre>	<pre> struct state{ uint8_t state; //00：写入成功，01：写入失败 } </pre>
15	NEED_ACK	读取EMS 端口子网掩码	<pre> struct port { uint8_t port; //1:ETH1, 2:ETH2, 3:EMS1, 4:EMS2 }; </pre>	<pre> struct in_addr { //小端字序 uint32_t s_addr; }; </pre>
16	UNNEED_ACK	A核控制字	struct BBMS_A_CtlWord	定时由A核发往M核
17	NEED_ACK	设置三级板打印级别	<pre> uint8_t printf_level; //取值来自于下面枚举 类型 typedef enum { TRACE_NONE = 0x0, // 不打印 TRACE_INFO = 0x01, // 打印信息及信息以 上的 TRACE_WARN = 0x02, // 只打印报警及报 警以上的 TRACE_ERRO = 0x04, //只打印错误及错 误以上的 TRACE_ALL = 0x07 // 全部打印 }TRACE_LEVEL ; </pre>	<pre> uint8_t state; //00：设置成功，01：设置失败 </pre>
18	NEED_ACK	读取三级板打印级别	NONE	<pre> uint8_t printf_level; //取值来源于下面的定义 typedef enum { TRACE_NONE= 0x0, //不打印 TRACE_INFO= 0x01, //只打印信息及信息 以上的 TRACE_WARN = 0x02, //只打印报警及报 警以上的 TRACE_ERRO = 0x04, //只打印错误及错误 以上的 TRACE_ALL = 0x07 //全部打印 }TRACE_LEVEL ; </pre> 获取打印级别失败返回255，需弹窗提示

19	UNNEED_ACK	发送A核RTC时间	<pre>struct A_Core_Rtc_time { uint16_t year; uint8_t month; uint8_t day; uint8_t hour; uint8_t min; uint8_t second; };</pre> 按此结构体进行发送	定时1s由A核发往M核
20	NEED_ACK	获取A核系统运行参数	<pre>{ uint8_t param_index; }</pre> ↵ param_index:系统参数index号 ↵ 1: A核拉M核 (M核复位) 次数 ↵ 2: A核核间通信发送速率, 单位: Byte/s 3: A核核间通信接收速率, 单位: Byte/s 4: PHY复位次数 5: 当前A核与二级板以太网成功连接簇数 6: 当前A核与二级板以太网成功连接状态 (64位无符号整型) bit0: Rank1以太网通讯状态 1: 通讯成功 0: 通讯失败 bit1: Rank2以太网通讯状态 1: 通讯成功 0: 通讯失败 7: A核检测二级板心跳状态 (64位无符号整型) bit0: Rank1 A核检测二级板心跳通讯状态 1: 心跳正常 0: 心跳异常 bit1: Rank2 A核检测二级板心跳通讯状态 1: 心跳正常 0: 心跳异常 8: M核检测二级板心跳状态 (64位无符号整型) bit0: Rank1 M核检测二级板心跳通讯状态 1: 心跳正常 0: 心跳异常 bit1: Rank2 M核检测二级板心跳通讯状态 1: 心跳正常 0: 心跳异常	<pre>{ uint8_t param_index; uint8_t data[]; //不定长数组 }</pre> ↵ param_index:系统参数index号 ↵ 1: A核拉M核 (M核复位) 次数 2: A核核间通信发送速率, 单位: Byte/s 3: A核核间通信接收速率, 单位: Byte/s 4: PHY复位次数 5: 当前与二级板以太网成功连接簇数 6: 当前与二级板以太网成功连接状态 (64位无符号整型) 7: A核检测二级板心跳状态 8: M核检测二级板心跳状态 ↵ data: 编号为param_index系统参数的数值 (大小任意, 为该系统参数的数据类型的大小) param_index:255,表示获取失败, data字段无数据。
21	NEED_ACK	设置报文是否加密, 并将此选项值写入到指定文件	<pre>uint8_t MsgEncryptMode ;</pre> 0: 数据不加密; 1: 数据开启加密	<pre>uint8_t state;</pre> //00: 设置成功并写入正确, 01: 设置失败或写入失败 //由上位机发送命令
22	NEED_ACK	请求从指定文件读取报文是否加密选项值	无	<pre>uint8_t MsgEncryptMode ;</pre> //00: 关闭加密, 01: 开启加密 //255: 获取失败。 //由上位机发送命令
23	NEED_ACK	设置N拖M三级板ID号 (只支持N托M架构使用)	<pre>uint8_t bbms_id;</pre>	<pre>uint8_t state;</pre> //00: 处理成功, 01: 处理失败

24	NEED_ACK	获取N托M三级板ID号	无	uint8_t bbms_id; ↵ //1-250: N托M三级板ID号 ↵ //255：获取失败
25	NEED_ACK	设置三级板主机/从机模式（只支持N托M架构使用）	uint8_t bbms_mode; //0x1：主机，0x2：从机	uint8_t state; //00：处理成功，01：处理失败
26	NEED_ACK	获取三级板（主机/从机）模式状态	无	uint8_t bbms_mode; //0x1：主机，0x2：从机（N托M架构才会返回2的数值，其它架构只会等于1） //255：获取失败
27	NEED_ACK	设置预制舱编号(箱号)	uint8_t data[];	uint8_t state; //00：处理成功，01：处理失败
28	NEED_ACK	获取预制舱编号(箱号)	无	uint8_t data[];
29	NEED_ACK	设置61850客户指定名称功能	uint8_t data[];	uint8_t state; //00：处理成功，01：处理失败
30	NEED_ACK	获取61850客户指定名称功能	无	uint8_t result; //00：读取成功，01：读取失败 uint8_t data[]; //读取成功后字符串数组
31	NEED_ACK	读取NTP IP地址列表	无	struct in_addr { uint8_t ntp_ip_num; //NTP IP地址数量,值为255时表示获取失败，且s_addr部分无内容 //小端字节序 // IPv4 subnet_address (in network byte order) uint32_t s_addr[ip_num]; //回读的IP地址（支持多个） };
32	NEED_ACK	NTP服务联调测试	无	uint8_t state; //00：测试正常，01：NTP服务器未联通

33	UNNEED_ACK	M核系统运行状态  (定时1s由M核发往A核)(核间通讯失联认为系统运行失败，一般被认为需要重启)	<pre>//M核运行状态 struct M_Core_System_Run_Status { uint8_t rbms_heart_loss[]; //M核检测二级板心跳丢失标志位,数组大小为系统总簇数 RACK_NUMBER_FOR EVERY_BANK 值, 值为1表示丢失 (3秒未收到rbms心跳报文表示丢失), 0表示正常;  uint8_t rbms_lan_can_select[]; //与二级板双网冗余LAN与CAN通道选择 (RBMS_CommType值一致), 数组大小为系统总簇数 RACK_NUMBER_FOR EVERY_BANK值 uint16_t rbms_can_send_speed[]; //与对应的二级板CAN通讯时三级板CAN报文发送速率,数组大小为系统总簇数即 RACK_NUMBER_FOR EVERY_BANK 值, 单位为Bytes/s uint16_t rbms_can_rcv_speed[]; //与对应的二级板CAN通讯时三级板CAN报文接收速率,数组大小为系统总簇数即 RACK_NUMBER_FOR EVERY_BANK 值, 单位为Bytes/s uint16_t hmi_can_send_speed; //与上位机CAN通讯时三级板CAN报文发送速率 uint16_t hmi_can_rcv_speed; //与上位机CAN通讯时三级板CAN报文接收速率 uint8_t rbms_can_comm_lost_count[]; //与对应的二级板CAN通讯时CAN报文通讯失联时间计数器,数组大小为系统总簇数即 RACK_NUMBER_FOR EVERY_BANK值 uint8_t bbms_task_stack_unused_ratio[32]; //各任务堆栈最小可用百分比 uint8_t bbms_task_used_cpu_ratio[32]; //各任务CPU占用率 (百分比) //任务ID定义如下, 与M核任务名对应 //TASK_ID_IDLE = 0, //TASK_ID_TMR_SVC = 1, //TASK_ID_MONITORTASK = 2, //TASK_ID_RPMSGSENDTASK = 3, //TASK_ID_RPMSGTASK = 4, //TASK_ID_RPMSGHANDLERTASK = 5, //TASK_ID_RETRYTASK = 6, //TASK_ID_CANFDMANAGERTASK = 7, //TASK_ID_YCPDECODETASK_0 = 8, //TASK_ID_NVMTASK = 9, //TASK_ID_IDLEARNTASK = 10, //TASK_ID_PROTOCOLTASK = 11, //TASK_ID_MBRTURS4851 = 12, //TASK_ID_MBRTURS4853OPERA = 13, //TASK_ID_MBRTURS4853 = 14, //TASK_ID_MBRTURS4854 = 15, //TASK_ID_CANTASK = 16, //TASK_ID_EXTDEVDATASENDTASK = 17, //TASK_ID_IOTASK = 18, //TASK_ID_ASW_10MTASK = 19,</pre>	无
----	------------	--	---	---

			<pre> //TASK_ID_ASW_100MSTASK = 20, //TASK_ID_ASW_1000MSTASK = 21, //TASK_ID_LEDTASK = 22, //TASK_ID_WDOGTASK = 23, uint32_t bbms_task_run_status; 按bit解析，当bit=1表示运行正常，bit=0表示运行异常（根据各任务心跳进行判断得出），位的含义如上定义。 uint8_t bbms_used_heap_unused_ratio; //目前M核FreeRTOS最小可用堆栈百分比 uint8_t bbms_freertos_used_cpu_ratio; //目前M核FreeRTOS系统CPU使用率（百分比） uint16_t canfd_route_boardcast_frame_speed; //目前M核CANFD路由广播帧发送速率，单位：帧数/秒 uint16_t rpmsg_route_boardcast_frame_speed; //目前M核核间通讯RPMSP路由广播帧发送速率，单位：帧数/秒 uint16_t rpmsg_send_speed; //M核核间通讯发送速率，单位为Bytes/s uint16_t rpmsg_rcv_speed; //M核核间通讯接收速率，单位为Bytes/s }; </pre>	
--	--	--	---	--

三、BMS2.0 BBMS_M业务传输命令（cmdGroup = 0x02）

三级板M核需要发送、接收并处理这些数据

命令号	命令类型	功能	请求payload	响应payload
1	UNNEED_ACK	BBMS概要数据	struct BBMS_SumInfo	
02	UNNEED_ACK	热管理信息	struct TMS_SumInfo	
03	NEED_ACK	HMI TMS控制指令	struct HMI_TMSCtrlWord	<pre> struct state{ uint8_t state; //00：写入成功，01：写入失败 } </pre>
04	NEED_ACK	HMI三级板DO控制指令（灯）	struct HMI_BankDOCtrl	<pre> struct state{ uint8_t state; //00：写入成功，01：写入失败 } </pre>
05	NEED_ACK	Bank累计充放电容量能量标定 此命令不可再被使用	struct HMI_BankAccuEnerg	<pre> struct state{ uint8_t state; //00：写入成功，01：写入失败 } </pre>
06	NEED_ACK	读取-TMS阈值参数		struct ParaThr_TMS

07	NEED_ACK	写入-TMS阈值参数	struct ParaThr_TMS	<pre> struct state{ uint8_t state; //00：写入成功，01：写入失败 } </pre>
08	UNNEED_ACK	外设数据	<pre> uint16_t reg_addr; //EMS点表地址 uint16_t peripheral_id; //外设id号 struct __attribute__((packed)) BBMSExternalDevices </pre>	reg_addr（三级板外设寄存器地址）范围：61600~65199 ↵ peripheral_id：外设id号 列表如下： 英维克液冷机组 外设id=1 安第斯除湿机外设 id=1001 三达除湿机外设 id=1002 同飞、英维克除湿机外设 id=1003 M2 ups外设 id=2001 CMC G2 ups外设 id=2002 CMC ups外设 id=2003 科士达ups外设 id=2004 U3-EC空调外设 id=3001 电表种类1外设 id=4001 消防种类1外设 id=5001
09	NEED_ACK	DIDO状态		<pre> struct __attribute__((packed)) IOState { uint8_t DI1:1; uint8_t DI2:1; uint8_t DI3:1; uint8_t DI4:1; uint8_t DI5:1; uint8_t DI6:1; uint8_t DI7:1; uint8_t DI8:1; uint8_t DI9:1; uint8_t DI10:1; uint8_t DI11:1; uint8_t DI12:1; uint8_t DO0:1; uint8_t DO1:1; uint8_t DO2:1; uint8_t DO3:1; uint8_t DO4:1; uint8_t DO5:1; uint8_t DO6:1; uint8_t DO7:1; uint8_t DO8:1; uint8_t DO9:1; uint8_t DO10:1; uint8_t DO11:1; }; </pre>
10	NEED_ACK	HMI三级板DO控制命令	struct HMI_BBMSDOCtrl	<pre> struct state{ uint8_t state; //00：写入成功，01：写入失败 } </pre>
11	NEED_ACK	写入-Bank故障累积次数标定	struct HMI_BankFaultCali	<pre> struct state{ uint8_t state; //00：写入成功，01：写入失败 } </pre>
12	UNNEED_ACK	存储所有Rack的BBMS控制命令	struct BBMS_CtlWordAllRack	

13	NEED_ACK	掉电通知	uint8_t state;//0:电源正常，1：电源异常	struct state{ uint8_t state; //00：接收成功 }
14	UNNEED_ACK	功能安全相关信号	struct BBMS_SafetySignal	
15	NEED_ACK	设置当前接入的除湿机485通讯地址（设置时只允许一台除湿机在线）	uint8_t dehumidifier_addr; //除湿机modbus通讯地址，该设置值只能为1或4	struct state{ uint8_t state; //0：设置成功 //1：设置的地址不合法 //2：设置失败 //3：除湿机不在线 //4：多台除湿机在线 }
16	NEED_ACK	读取当前接入的除湿机485通讯地址（读取时只允许一台除湿机在线）	无	struct dehumidifier_addr{ int16_t dehumidifier_addr; //除湿机485通讯地址 //1~247：正确地址（目前地址仅为1或4） //1：告警“除湿机不在线” //2：告警“多台除湿机在线” //3：读取失败 }
17	NEED_ACK	A核设置M核打印日志级别（仅在调试M核程序时使用，RS485_4被占用时（目前RS485_4外接电表或UPS)无法使用）	uint8_t printf_level; //与三级板打印级别的定义一致	uint8_t state; //00: 设置成功，01：设置失败
18	NEED_ACK	A核读取M核打印日志级别（仅在调试M核程序时使用，RS485_4被占用时（目前RS485_4外接电表或UPS)无法使用）	NONE	uint8_t printf_level; //与三级板打印级别的定义一致
19	UNNEED_ACK	BBMS故障列表	struct BBMS_Fault	

20	NEED_ACK	设置M核CAN报文是否加密	uint8_t MsgEncryptMode ; 0: 数据不加密； 1：数据开启加密	uint8_t state; //00：设置成功，01：设置失败 //由A核发送命令
21	NEED_ACK	获取M核CAN报文是否加密	无	uint8_t MsgEncryptMode ; //00:关闭加密，01：开启加密 //255:获取失败。 //由A核发送命令
22	UNNEED_ACK	A核系统运行状态 ← (定时1s由A核发往M核)(核间通讯失联认为系统运行失败，一般被认为需要重启)	struct A_Core_System_Run_Status { uint8_t rbms_heart_loss[]; //A核检测二级板心跳丢失标志位,数组大小为系统总簇数 RACK_NUMBER_FOR_EVERY_BANK值，值为1表示丢失（3秒未收到rbms心跳报文表示丢失），0表示正常； uint8_t rbms_lan_can_select[]; //与二级板双网冗余LAN与CAN通道选择 (RBMS_CommType值一致)，数组大小为系统总簇数 RACK_NUMBER_FOR_EVERY_BANK值 }	无

四、BMS2.0 RBMS业务传输命令（cmdGroup = 0x03）

二级板需要发送、接收以及处理这些数据

命令号	命令类型	功能	请求payload	响应payload
1	UNNEED_ACK	RBMS概要数据	struct RBMS_SumInfo	
2	UNNEED_ACK	单体电压及有效性 电芯电压及其有效性、各AFE总压	struct RBMS_CellV struct RBMS_Volt	
3	UNNEED_ACK	模组温度及有效性 电芯温度、极柱温度、Pack正负极连接件温度、PCB板温及其有效性	struct RBMS_CellTemp struct RBMS_Temp	
4	UNNEED_ACK	电芯均衡状态	struct RBMS_CellBalSt	
5	UNNEED_ACK	单体电芯自放电率	struct RBMS_CellSdr	
6	NEED_ACK	设置Rack ID	int id;	struct state{ uint8_t state; //00：写入成功，01：写入失败 }

07	NEED_ACK	BBMS控制指令	struct BBMS_CtlWord	struct state{ uint8_t state; //00：写入成功，01：写入失败 }
08	NEED_ACK	Rack标定与控制	struct HMI_RackCaliCtrl	struct state{ uint8_t state; //00：写入成功，01：写入失败 }
09	NEED_ACK	HMI二级板继电器强 控指令	struct HMI_RBMSRlyCtrl	struct state{ uint8_t state; //00：写入成功，01：写入失败 }
10	NEED_ACK	读取-电压阈值参数		struct ParaThr_CellV
11	NEED_ACK	写入-电压阈值参数	struct ParaThr_CellV	struct state{ uint8_t state; //00：写入成功，01：写入失败 }
12	NEED_ACK	读取-簇电压阈值参数		struct ParaThr_RackV
13	NEED_ACK	写入-簇电压阈值参数	struct ParaThr_RackV	struct state{ uint8_t state; //00：写入成功，01：写入失败 }
14	NEED_ACK	读取-簇电流阈值参数		struct ParaThr_RackI
15	NEED_ACK	写入-簇电流阈值参数	struct ParaThr_RackI	struct state{ uint8_t state; //00：写入成功，01：写入失败 }
16	NEED_ACK	读取-温度阈值参数		struct ParaThr_ModuleT
17	NEED_ACK	写入-温度阈值参数	struct ParaThr_ModuleT	struct state{ uint8_t state; //00：写入成功，01：写入失败 }
18	NEED_ACK	读取-SOX阈值参数		struct ParaThr_SOX
19	NEED_ACK	写入-SOX阈值参数	struct ParaThr_SOX	struct state{ uint8_t state; //00：写入成功，01：写入失败 }
20	NEED_ACK	读取-AUX阈值参数		struct ParaThr_AUX

21	NEED_ACK	写入-AUX阈值参数	struct ParaThr_AUX	<pre> struct state{ uint8_t state; //00：写入成功，01：写入失败 } </pre>
22	NEED_ACK	获取DIDO状态	struct RBMS_IOState	<pre> struct __attribute__((packed)) RBMS_IOState { uint8_t DI_DIL_IN1:1; uint8_t DI_DIH_IN2:1; uint8_t DI_DIL_IN3:1; uint8_t DI_DIH_IN4:1; uint8_t DI_DIL_IN5:1; uint8_t DI_DIH_IN6:1; uint8_t DI_DI_IN7:1; uint8_t DI_DI_IN8:1; uint8_t DO_HSD_IN1:1; uint8_t DO_HSD_IN2:1; uint8_t DO_HSD_IN3:1; uint8_t DO_HSD_IN4:1; uint8_t DO_HSD_IN5:1; uint8_t DO_HSD_IN6:1; uint8_t DO_HSD_IN7:1; uint8_t DO_HSD_IN8:1; uint8_t DO_LSD_EN1:1; uint8_t DO_LSD_EN2:1; uint8_t DO_LSD_EN3:1; uint8_t DO_LSD_EN4:1; uint8_t DO_LSD_EN5:1; uint8_t DO_LSD_EN6:1; uint8_t DO_LSD_EN7:1; uint8_t DO_LSD_EN8:1; }; </pre>
23	UNNEED_ACK	PCB板温信息 模型调试信息	<pre> struct RBMS_PCBBdTemp struct RBMS_Debug </pre>	
24	NEED_ACK	HMI二级板DO控制命令	struct HMI_RBMSDOCtrl	<pre> struct state{ uint8_t state; //00：写入成功，01：写入失败 } </pre>
25	UNNEED_ACK	SOX算法调试的输入数据	struct RBMS_SOXdebugData1	
26	UNNEED_ACK	SOX算法调试的输出数据	struct RBMS_SOXdebugData2	
27	UNNEED_ACK	Rack中每个AFE的总压	struct RBMS_AFEVolt	
28	NEED_ACK	写入-Rack故障累积次数标定	struct HMI_RackFaultCali	<pre> struct state{ uint8_t state; //00：写入成功，01：写入失败 } </pre>
29	UNNEED_ACK	极柱温度	struct RBMS_PoleTemp	

30	UNNEED_ACK	需要进行主动均衡板编码的信息	<pre>typedef struct balancePosition { uint8_t rackId; /* rack ID */ uint8_t packId; /* pack ID */(packid每个位表示哪个pack 需要进行编码) };</pre>	
31	UNNEED_ACK	启动该簇进行位置校验的命令（二级板接到命令就表示开始进行位置校验逻辑）	无	
32	UNNEED_ACK	Pack正负极连接件温度	<pre>struct RBMS_PackPosNegConnTemp</pre>	
33	NEED_ACK	该簇PCAK下线测试模式开关命令	cmd	cmd = 0； 关闭PACK下线测试模式 cmd = 1； 打开PACK下线测试模式
34	UNNEED_ACK	外设数据	<pre>struct __attribute__((packed)) RBMSEExternalDevices</pre>	
35	NEED_ACK	设置二级板log打印等级	<pre>typedef enum { TRACE_NONE, //不打印 TRACE_INFO=0x01, //只打印信息 TRACE_WARN=0x02, //只打印报警 TRACE_ERRO=0x04, //只打印错误 TRACE_ALL = 0x07, //全部打印 }TRACE_LEVEL;</pre>	//00：设置成功，01：设置失败
36	NEED_ACK	读取二级板log打印等级	NONE	<pre>typedef enum { TRACE_NONE, //不打印 TRACE_INFO=0x01, //只打印信息 TRACE_WARN=0x02, //只打印报警 TRACE_ERRO=0x04, //只打印错误 TRACE_ALL = 0x07, //全部打印 }TRACE_LEVEL;</pre>

37	NEED_ACK	HMI TMS控制指令 (RBMS)	struct HMI_TMSCtrlWord	struct state{ uint8_t state; //00：写入成功，01：写入失败 }
38	UNNEED_ACK	热管理信息(RBMS)	struct TMS_SumInfo	
39	NEED_ACK	读取-TMS阈值参数 (RBMS)		struct ParaThr_TMS
40	NEED_ACK	写入-TMS阈值参数 (RBMS)	struct ParaThr_TMS	struct state{ uint8_t state; //00：写入成功，01：写入失败 }
41	UNNEED_ACK	RBMS故障列表	struct RBMS_BBMS	

五、业务通用命令（cmdGroup = 0x04）

BMS系统（包括二级板和三级板AIM）需要接收并处理这些数据

命令号	命令类型	功能	请求payload	响应payload
1	UNNEED_ACK	错误码	struct FaultFlags	
2	NEED_ACK	设置IP	struct in_addr { //小端字序 uint32_t s_addr; // IPv4 address (in network byte order) };	struct state{ uint8_t state; //00：写入成功，01：写入失败 }
3	NEED_ACK	读取IP		struct in_addr { //小端字序 uint32_t s_addr; // IPv4 address (in network byte order) };
4	NEED_ACK	HMI 控制指令	struct HMI_CtrlWord	uint8_t state; //00：写入成功，01：写入失败
5	NEED_ACK	HMI-Logger广播信号 读取-RBMS/BBMS故障超限次数	struct HMI_BroadcastSig	uint8_t state; //00：写入成功，01：写入失败 struct HMI_FltOvTiNbr
6	NEED_ACK	清除NVM		uint8_t state; //00：清除成功，01：清除失败
7	NEED_ACK	启动/禁止设备主动发送报文	uint8_t ActiveSend; // 0：允许发送，1：禁止发送	uint8_t state; //00：操作成功
8	NEED_ACK	读取-故障使能数组		struct FaultFlags

9	NEED_ACK	写入-故障使能数组	struct FaultFlags	struct state{ uint8_t state; //00：写入成功，01：写入失败 }
10	NEED_ACK	设置子网掩码	struct ip4_addr { u32_t addr; }; //ip地址和子网掩码数据一样	struct state{ uint8_t state; //00：写入成功，01：写入失败 }
11	NEED_ACK	读取子网掩码		struct ip4_addr { u32_t addr; }; //ip地址和子网掩码数据一样

六、BMS2.0主动均衡板业务传输命令（cmdGroup = 0x05）

主动均衡板需要发送、接收这些数据

命令号	命令类型	功能	请求payload	响应payload
1	UNNEED_ACK	主动均衡板编址信息设置（上位机发送给主动均衡板）	typedef struct activeBalance_idState { uint8_t configState; /* 配置状态 0：表示清除配置，1：表示设置配置 */ uint8_t rackId; /* rack ID configState = 1设置的时候固定为默认配置0 */ uint8_t packId; /* pack ID configState = 1设置的时候固定为默认配置0 */ }activeBalance_idState_t;	
2	UNNEED_ACK	主动均衡板编址状态信息(主动均衡板发送给上位机)	typedef struct activeBalance_idState { uint8_t configState; /* 配置状态 0：没有编码状态，1：表示已编码状态 */ uint8_t rackId; /* rack ID */ uint8_t packId; /* pack ID */ }activeBalance_idState_t;	
3	UNNEED_ACK	二级板通知主动均衡板记录电压命令	无	
4	UNNEED_ACK	主动均衡板记录电压完成回复指令	无	

5	UNNEED_ACK	二级板启动均衡后告知主动均衡板可以进行比较命令	无	
6	UNNEED_ACK	主动均衡板发送给上位机校验结果	checkFlag: ↩ 1：位置不一致 0：位置一致	

.